

Proyecto: Optimización Metaheurística con Simulated Annealing y Algoritmos Genéticos

Fundamentos de Inteligencia Artificial — Universidad Anáhuac México

Prof. Dr. Aboud Barsekh-Onji

Abril 2026

Objetivo del Proyecto. Ampliar un script base de **Simulated Annealing (SA)** para aplicarlo a cuatro funciones de prueba clásicas en optimización, y luego implementar el mismo experimento usando **Algoritmos Genéticos (GA)**. Al finalizar, el alumno será capaz de adaptar algoritmos metaheurísticos a funciones arbitrarias, visualizar paisajes de optimización y comparar el desempeño de distintas metaheurísticas.

Archivos de Partida

La carpeta del proyecto contiene los siguientes archivos que ya están implementados:

Archivo	Descripción
<code>main_SA.m</code>	Script principal de Simulated Annealing. Funciona con la función Ackley.
<code>Ackley.m</code>	Función Ackley (fuente: SFU Optimization Test Functions).

Ejecuta `main_SA.m` en MATLAB **antes de cualquier modificación** para verificar que funciona. Debes obtener un valor de función cercano a $f^* = 0$ con $x^* \approx (0, 0)$.

Parte 1 — Visualización de la Función Ackley

Tarea 1.1 — Gráfica de superficie. Crea un script `plot_ackley.m` que genere la gráfica de superficie 3D y el mapa de contorno de la función Ackley en 2D sobre el dominio $[-5, 5]^2$. Usa `surf`, `contourf` y `shading interp`.

Guía: El script `main_SA.m` ya genera un contorno en la subgráfica 4 cuando `d = 2`. Inspírate en esa sección para construir `plot_ackley.m`.

Preguntas a responder en el reporte:

1. ¿Por qué la función Ackley es un reto para los optimizadores? Identifica en la gráfica el mínimo global y los mínimos locales.
2. ¿Qué relación tiene la temperatura inicial T_0 del SA con el paisaje de la función?

Parte 2 — Descarga y Adaptación de Nuevas Funciones

Descarga los scripts MATLAB del repositorio de funciones de prueba de la Universidad Simon Fraser:

<https://www.sfu.ca/~ssurjano/optimization.html>

Necesitas las siguientes tres funciones. Para cada una, navega a su página, descarga el archivo .m y colócalo en la misma carpeta que main_SA.m.

2.1 Bukin Function N. 6

- **Página:** <https://www.sfu.ca/~ssurjano/bukin6.html>
- **Archivo a descargar:** bukin6.m
- **Dimensión:** $d = 2$ únicamente
- **Dominio:** $x_1 \in [-15, -5]$, $x_2 \in [-3, 3]$ (dominio asimétrico)
- **Óptimo global:** $f(-10, 1) = 0$
- **Fórmula:**

$$f(\mathbf{x}) = 100\sqrt{|x_2 - 0.01x_1^2|} + 0.01|x_1 + 10|$$

Nota importante: los límites de búsqueda son **diferentes** para x_1 y x_2 . El vector `lb` y `ub` en `main_SA.m` debe reflejar esto: `lb = [-15, -3]`, `ub = [-5, 3]`.

2.2 Levy Function

- **Página:** <https://www.sfu.ca/~ssurjano/levy.html>
- **Archivo a descargar:** levy.m
- **Dimensión:** cualquier $d \geq 1$ (usa $d = 2$ para comparar, $d = 10$ como experimento extra)
- **Dominio:** $x_i \in [-10, 10]$
- **Óptimo global:** $f(1, \dots, 1) = 0$
- **Fórmula:** sea $w_i = 1 + \frac{x_i - 1}{4}$, entonces

$$f(\mathbf{x}) = \sin^2(\pi w_1) + \sum_{i=1}^{d-1} (w_i - 1)^2 \left[1 + 10 \sin^2(\pi w_{i+1}) \right] + (w_d - 1)^2 \left[1 + \sin^2(2\pi w_d) \right]$$

2.3 Rastrigin Function

- **Página:** <https://www.sfu.ca/~ssurjano/rastr.html>
- **Archivo a descargar:** rastr.m
- **Dimensión:** cualquier $d \geq 1$ (usa $d = 2$ para comparar, $d = 10$ como experimento extra)
- **Dominio:** $x_i \in [-5.12, 5.12]$
- **Óptimo global:** $f(0, \dots, 0) = 0$

- **Fórmula:**

$$f(\mathbf{x}) = 10d + \sum_{i=1}^d [x_i^2 - 10 \cos(2\pi x_i)]$$

Parte 3 — SA sobre las Cuatro Funciones

Tarea 3.1 — Adaptar `main_SA.m` para cada función. Para cada una de las tres funciones descargadas, crea una copia de `main_SA.m` con el nombre indicado y modifica **únicamente la sección** marcada con `>> CONFIGURAR AQUÍ <<`.

Script a crear	Función	FUN_NAME	d	lb	ub
<code>main_SA_bukin6</code>	Bukin N.6	'bukin6'	2	[-15, -3]	[-5, 3]
<code>main_SA_levy</code>	Levy	'levy'	2	[-10, -10]	[10, 10]
<code>main_SA_rastrigin</code>	Rastrigin	'rastr'	2	[-5.12, -5.12]	[5.12, 5.12]

Cambios mínimos requeridos en cada copia (el resto del script queda intacto):

```
% >>> CONFIGURAR AQUÍ <<<
FUN_NAME      = 'bukin6';
objective_fun  = @(x) bukin6(x); % <-- cambia al nombre de tu función

d = 2;
lb = [-15, -3]; % <-- límites inferiores
ub = [-5, 3]; % <-- límites superiores
```

Tarea 3.2 — Ajuste de hiperparámetros. El SA tiene tres parámetros críticos: `T0`, `alpha` y `sigma`. Para cada función, experimenta con al menos dos configuraciones distintas y reporta cuál dio mejor resultado. Guía de ajuste:

- Funciones muy multimodales (Bukin, Rastrigin): necesitan `T0` más alto y `alpha` cercano a 1 (0.995–0.999).
- Funciones más suaves (Levy): pueden funcionar bien con `T0` menor.
- Si converge demasiado rápido a un mal mínimo: reduce `alpha` (enfría más lento).

Tarea 3.3 — Tabla comparativa. Ejecuta cada script **10 veces** (usa semillas del 1 al 10: `rng(1)`, `rng(2)`, ...) y completa la tabla siguiente en tu reporte.

Función	f^* teórico	Media f_{best}	Desv. est.	Mejor f_{best}	Peor f_{best}
Ackley	0				
Bukin N.6	0				
Levy	0				
Rastrigin	0				

Función	f^* teórico	Media f_{best}	Desv. est.	Mejor f_{best}	Peor f_{best}
---------	---------------	------------------	------------	------------------	-----------------

Pregunta: ¿Qué función resultó más difícil para el SA? ¿Por qué?

Parte 4 — Algoritmos Genéticos sobre las Mismas Funciones

Tarea 4.1 — Script main_GA.m. Crea main_GA.m usando la función `ga()` del **MATLAB Optimization Toolbox** (disponible en R2025b). El script debe seguir la misma filosofía modular que main_SA.m: una sección `>> CONFIGURAR AQUÍ <<<` donde se cambia la función, la dimensión y los límites.

Estructura sugerida para main_GA.m:

```
% =====
% main_GA.m - Algoritmo Genético sobre función arbitraria
% =====

clear; clc; close all;

% >>> CONFIGURAR AQUÍ <<<
FUN_NAME      = 'ackley';
objective_fun = @(x) ackley(x);
d             = 2;
lb            = -32.768 * ones(1, d);
ub            = 32.768 * ones(1, d);

% --- Parámetros del GA ---
options = optimoptions('ga', ...
    'PopulationSize', 200, ... % tamaño de la población
    'MaxGenerations', 500, ... % generaciones máximas
    'EliteCount',     5, ... % individuos élite (no mutan)
    'CrossoverFraction', 0.8, ... % fracción de cruzamiento
    'MutationFcn', @mutationgaussian, ...
    'PlotFcn', @gaplotbestf, ...
    'Display', 'iter');

% --- Ejecución del GA ---
[x_best, f_best] = ga(objective_fun, d, [], [], [], [], lb, ub, [], options);

% --- Resultados ---
fprintf('Mejor valor GA : f = %.8f\n', f_best);
fprintf('Mejor posición : x = ');
fprintf('%.4f ', x_best);
fprintf(']\n');
```

Tarea 4.2 — GA para las cuatro funciones. Crea cuatro scripts (uno por función) modificando solo la sección de configuración. Completa la misma tabla comparativa que en la Tarea 3.3, ahora para el GA.

Tarea 4.3 — Comparación SA vs. GA. Genera una gráfica de barras comparando la media del error absoluto $|f_{best} - f^*|$ de SA y GA para cada función. Responde en el reporte:

1. ¿Cuál algoritmo convergió mejor en cada función?
2. ¿Cuál fue más rápido en tiempo de cómputo?
3. ¿Qué parámetro del GA tiene mayor impacto sobre la calidad del resultado?

Parte 5 — Experimento de Alta Dimensión (Extra)

Tarea 5 (Extra) — SA y GA en $d = 10$. Repite el experimento de las Partes 3 y 4 para la función **Rastrigin** con $d = 10$ dimensiones. ¿Cómo cambia la dificultad del problema? ¿Cuál algoritmo escala mejor?

Entregables

Qué debes entregar (archivo .zip):

1. Scripts MATLAB:

- main_SA.m (original o con ajustes documentados)
- main_SA_bukin6.m, main_SA_levy.m, main_SA_rastrigin.m
- main_GA.m, main_GA_bukin6.m, main_GA_levy.m, main_GA_rastrigin.m
- plot_ackley.m
- Funciones descargadas: Ackley.m, bukin6.m, levy.m, rastr.m

2. Reporte en PDF que incluya:

- Respuestas a las preguntas de cada sección
- Gráficas de convergencia del SA y del GA para cada función (1 figura por función)
- Tablas comparativas de la Tarea 3.3 y 4.2
- Gráfica de barras SA vs. GA (Tarea 4.3)
- Conclusión de media página: ¿qué aprendiste sobre la selección de metaheurísticas?

Criterios de Evaluación

Criterio	Puntos
Código funcional: SA en las 4 funciones	30
Código funcional: GA en las 4 funciones	25
Visualización de funciones y convergencia	15
Análisis y tablas comparativas	20
Conclusión y calidad del reporte	10
Total	100

Referencias

- Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P. (1983). *Optimization by Simulated Annealing*. **Science**, 220(4598), 671–680.
- Holland, J.H. (1992). *Adaptation in Natural and Artificial Systems*. MIT Press.
- Surjanovic, S. & Bingham, D. (2013). *Virtual Library of Simulation Experiments: Test Functions and Datasets*. Simon Fraser University. <https://www.sfu.ca/~ssurjano/optimization.html>
- MathWorks (2025). *ga — Find minimum of function using genetic algorithm*. MATLAB R2025b Documentation.